HW1: LunarLander — Deep Q-Network & Double DQN

PKU Reinforcement Learning Course

MAK CHAK WING UID: 2501213361

2026

Contents

1 Environment & Problem Setting **2**

2 DQN Method **2**

3 DDQN Method **3**

4 Implementation Details & Hyperparameters **4**

5 Training Results **5**

5.1 Task 1 — DQN 5

5.2 Task 2 — DDQN 6

6 DQN vs. DDQN Comparison **8**

7 Conclusion **8**

1 Environment & Problem Setting

LunarLander-v3 is a continuous-state, discrete-action control task in the Gymnasium library. The goal is to land the lunar module safely on the pad between the two flags.

State space (8-dimensional, continuous):

- Horizontal and vertical position (x, y)
- Horizontal and vertical velocity ($\dot{x}$, $\dot{y}$)
- Lander angle θ and angular velocity $\dot{\theta}$
- Two binary leg contact indicators (left, right)

Action space (4 discrete actions):

Action	Description
0	Do nothing
1	Fire left engine
2	Fire main engine
3	Fire right engine

Reward structure. The environment uses shaped rewards to guide learning. A successful landing gives about +100–140, crashing gives −100, each leg contact gives +10, and using the main engine costs −0.3 per frame. The task is considered *solved* when the cumulative reward is above 200.

Since the state space is continuous and fairly high-dimensional, tabular methods such as Q-learning are not practical here. Instead, a neural network is used to approximate the Q-function.

2 DQN Method

Deep Q-Network (DQN) extends Q-learning to continuous state spaces by replacing the Q-table with a neural network $Q(s, a; \theta)$. In this implementation, training is stabilized by three standard components.

Neural network Q-function.
A two-layer MLP with ReLU activations is used:

$$\text{Input}(8) \;\rightarrow\; \text{FC}(128) \;\rightarrow\; \text{FC}(128) \;\rightarrow\; \text{Output}(4).$$

The network outputs one Q-value for each action, and the value of the selected action is updated toward the Bellman target using Huber (SmoothL1) loss.

Experience replay.
Transitions $(s, a, r, s', \mathit{done})$ are stored in a circular replay buffer with capacity 100,000. During

training, a mini-batch of 64 transitions is sampled uniformly at each update step. This helps reduce the correlation between consecutive samples and improves sample efficiency.

Target network.
A separate target network with parameters θ^- is used to compute stable regression targets:

$$y = r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-).$$

The target network is updated by copying the online network every 1,000 gradient steps. Without this, the target changes too quickly and training can become unstable.

ε-greedy exploration.
The agent follows an ε-greedy policy. It chooses a random action with probability ε, which decays multiplicatively from $\varepsilon_0 = 1.0$ to $\varepsilon_{\min} = 0.01$ using a factor of $\times 0.995$ per episode. This allows more exploration at the beginning and gradually shifts the policy toward greedy action selection.

3 DDQN Method

A known issue in standard DQN is that it can overestimate Q-values, because the same network is used for both action *selection* and action *evaluation*. Double DQN (DDQN) reduces this problem by separating these two roles.

Algorithm	TD Target
DQN	$r + \gamma \cdot \max_{a'} Q_{\text{tgt}}(s', a')$
DDQN	$r + \gamma \cdot Q_{\text{tgt}}(s',\ \arg\max_{a'} Q_{\text{onl}}(s', a'))$

In DDQN, the *online* network chooses the greedy action, while the *target* network evaluates that action. In practice, this gives more conservative and usually more reliable value estimates. The nice part is that DDQN only changes the target computation, so the rest of the implementation stays the same.

4 Implementation Details & Hyperparameters

Table 1 lists the hyperparameters used in both Task 1 (DQN) and Task 2 (DDQN). The settings are the same except for the target computation formula.

Table 1: Hyperparameters used for both DQN and DDQN.

Parameter	Value
Environment	LunarLander-v3
Training episodes	1,000
Optimizer	Adam
Learning rate α	1×10^{-4}
Discount factor γ	0.99
Replay buffer capacity	100,000
Mini-batch size	64
ε (start / end / decay)	1.0 / 0.01 / $×$0.995 per episode
Target network update	Hard copy every 1,000 steps
Network architecture	FC(128)–ReLU–FC(128)–ReLU, input 8, output 4
Loss function	Huber (SmoothL1)
Gradient clipping	ℓ_2 norm ≤ 10
Random seed	42

5 Training Results

5.1 Task 1 — DQN

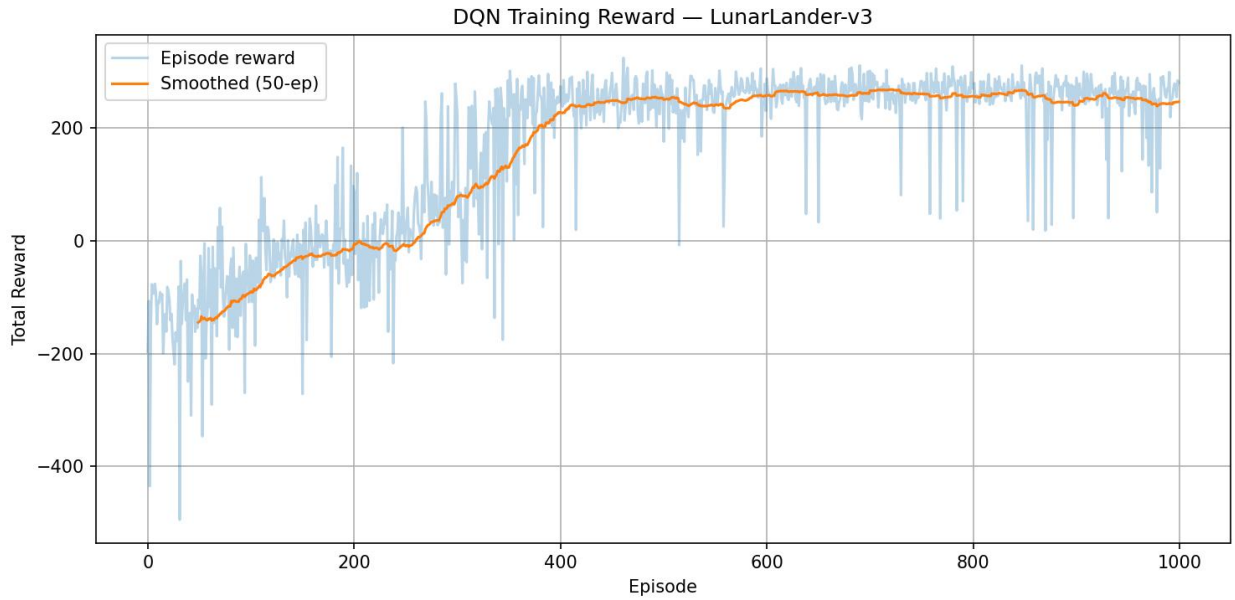

Figure 1: DQN training reward per episode (light blue) and 50-episode smoothed average (orange). The agent consistently solves the environment (reward ≥ 200) from around episode 400 onward.

From the reward curve, the smoothed return becomes positive at around episode 200, reaches about 230 near episode 400, and then stays above 240 for the rest of training. The final 50-episode average reward is ≈ 247, which is comfortably above the solve threshold of 200.

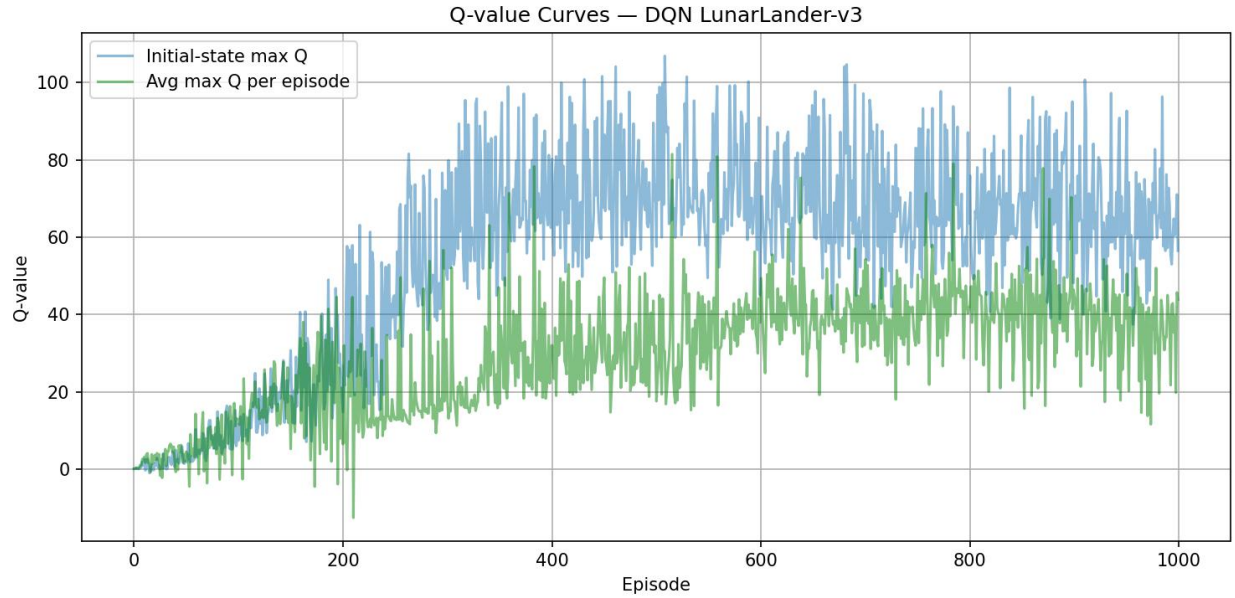

Figure 2: DQN Q-value curves: initial-state max Q (blue) and average max Q per episode (green).

Both Q-value curves increase over time, which matches the improvement in policy quality. The initial-state max Q (blue) stays higher than the per-episode average (green). One possible reason is that the initial state is relatively favorable: the lander starts at a reasonable altitude with full fuel, so the estimated return from that state is higher.

5.2 Task 2 — DDQN

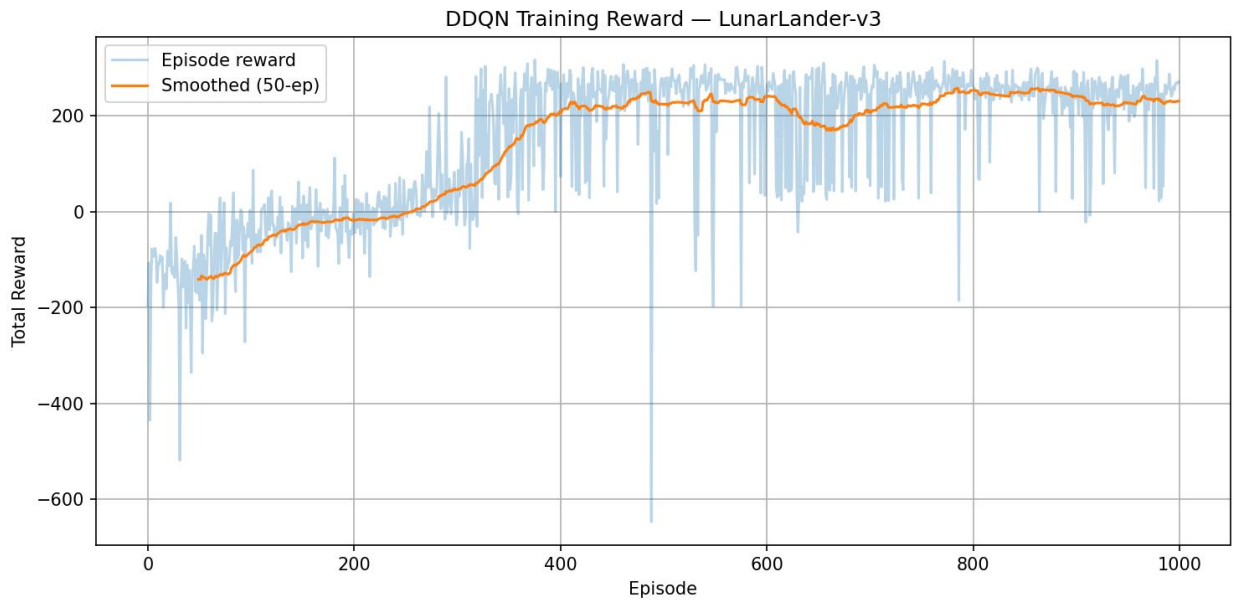

Figure 3: DDQN training reward per episode (light blue) and 50-episode smoothed average (orange).

DDQN shows a very similar learning trend. The smoothed reward passes 200 at around episode 420 and the final 50-episode average is ≈ 231. There is somewhat higher variance in the middle part of training, especially around episodes 500–700, but the method still converges.

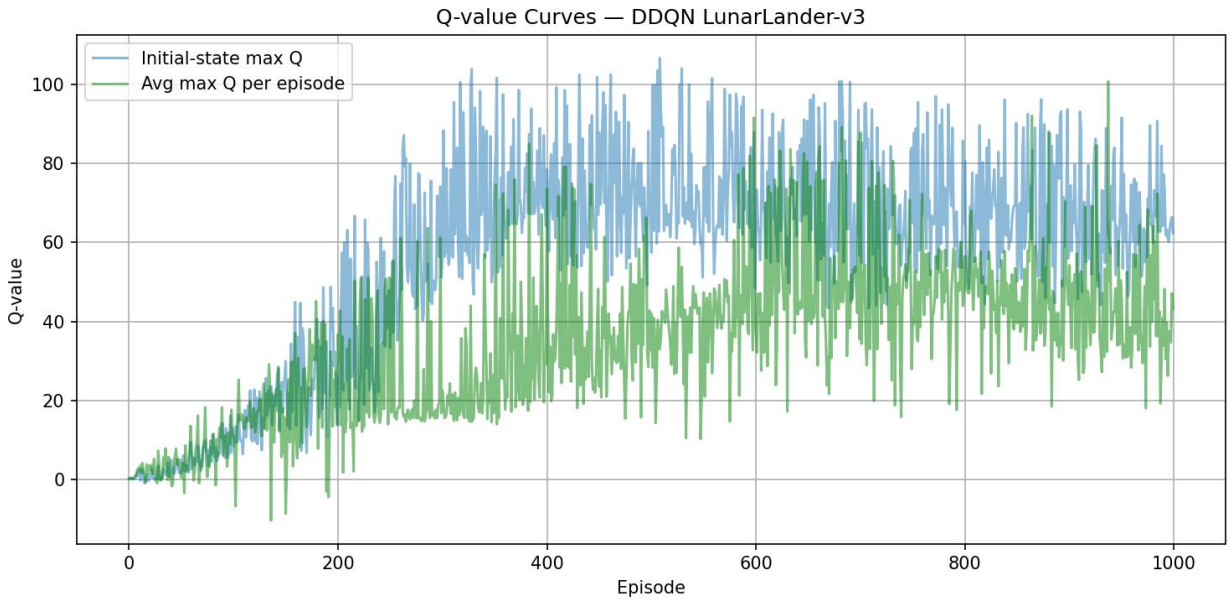

Figure 4: DDQN Q-value curves: initial-state max Q (blue) and average max Q per episode (green).

Compared with DQN, the DDQN Q-value curves grow a bit more steadily and show less episode-to-episode fluctuation. This matches the idea that DDQN reduces overestimation bias. The two curves are also closer to each other, which suggests that the value estimates are more consistent across the trajectory.

6 DQN vs. DDQN Comparison

Table 2: Side-by-side comparison of DQN and DDQN on LunarLander-v3.

Metric	DQN	DDQN
Final avg. reward (last 50 ep.)	≈ 247	≈ 231
Greedy test episode reward	≈ 241	≈ 293
Episodes to first reach reward ≥ 200	≈ 400	≈ 420
Peak Q-value (initial state)	≈ 100	≈ 100

Both methods solve the environment successfully. DQN converges a little faster and has a slightly higher average reward during training, while DDQN gets the better greedy test score (≈ 293 vs. ≈ 241). One possible explanation is the usual overestimation effect in DQN: during training, larger Q-values can make the policy look better than it really is, but once exploration is removed, the greedy policy may not be as strong as the training curve suggests. DDQN gives more conservative estimates, which seems to help test-time performance here.

The Q-value plots support this interpretation. DDQN keeps the value estimates smoother and slightly more controlled in magnitude. In tasks with larger action spaces or sparser rewards, where overestimation matters more, the gap between DDQN and DQN would likely be more obvious.

7 Conclusion

Both DQN and DDQN are able to solve LunarLander-v3 within 1,000 episodes on CPU. A few main observations from this homework are listed below:

- **Experience replay and a target network** are important for stable training. Without them, updates become too correlated and the target changes too quickly.
- **The DDQN change is small but useful**. Only the target computation is modified, but this is enough to reduce overestimation bias and improve greedy test performance without extra hyperparameter tuning.
- **The chosen hyperparameters work reasonably well** for both methods. A linear ε-decay schedule or prioritized replay could be possible directions for improving sample efficiency.
- **The most unstable stage is the early training phase**. Rewards stay below -200 during roughly the first 100–200 episodes, before the replay buffer becomes more diverse and ε decreases enough to reduce random behavior.